

PROJECT REPORT

REAL-TIME PUBLIC TRANSPORT TRACKING FOR SMALL CITIES

Submitted By

VIKASHINI P.

II Year – Department of Information Technology

VSB ENGINEERING COLLEGE, KARUR

Academic Year

2026 – 2027

1. PROJECT OVERVIEW

The Real-Time Public Transport Tracking for Small Cities is a web-based application developed to provide useful public transportation information to users. The system allows users to view available buses, driver details, routes, and bus location information.

The application consists of a frontend, Flask backend, REST API, and MySQL database. The backend handles the application logic and communicates with the MySQL database for storing and retrieving bus and route information.

The system implements Create, Read, Update, and Delete (CRUD) operations for bus management. REST APIs are tested using Postman to verify that the operations work correctly. The database is also verified using MySQL.

The project aims to provide a simple and efficient solution for public transport management and tracking, particularly for small cities.

2. PROBLEM STATEMENT

Public transportation is an important part of daily travel, especially in small cities. However, passengers may face difficulties in getting information about available buses, routes, drivers, and the current location of buses.

In many cases, users do not have an easy way to know where a particular bus is located or to access updated bus information. This can result in waiting time and difficulty in planning journeys.

Therefore, there is a need for a simple web-based system that can manage bus and route information and provide bus location details to users.

The proposed Real-Time Public Transport Tracking for Small Cities addresses this problem by providing a centralized system for managing bus information, routes, driver details, and location coordinates. The system also provides REST APIs for performing CRUD operations and uses a MySQL database for storing the information.

3. OBJECTIVES

The main objectives of the Real-Time Public Transport Tracking for Small Cities project are:

1. To develop a web-based system for managing public transport information.
2. To allow users to view available buses and their details.
3. To manage bus information such as bus number, driver name, and route details.
4. To provide bus location information using latitude and longitude coordinates.
5. To implement Create, Read, Update, and Delete (CRUD) operations for bus management.
6. To develop REST APIs for communication between the application and database.
7. To store and manage bus and route information using a MySQL database.
8. To validate user inputs and handle errors appropriately.
9. To test the REST APIs using Postman.
10. To provide a simple and efficient public transportation management solution for small cities.

4. SCOPE

The scope of the Real-Time Public Transport Tracking for Small Cities project includes the development of a complete web-based public transport management and tracking system.

The major areas covered by the project are:

- **User Interface:** Provides a simple interface for users to access public transport information.

- **Bus Management:** Allows bus details such as bus number, driver name, and route ID to be managed.
- **Route Management:** Stores and manages source, destination, and route-related information.
- **Location Tracking:** Stores the latitude and longitude coordinates of buses to represent their current location.
- **REST API:** Provides API endpoints for performing CRUD operations on bus records.
- **Database Integration:** Uses MySQL to store and retrieve application data.
- **Validation and Error Handling:** Handles invalid inputs and unavailable bus records using appropriate responses.
- **API Testing:** Uses Postman to test and verify the REST API operations.

The project can be further extended with features such as real-time GPS tracking, ETA calculation, crowd-level information, and notifications.

5. TECHNOLOGY STACK

Technology	Purpose
HTML	Used to create the structure of the web pages.
CSS	Used to design and style the user interface.
JavaScript	Used for client-side interaction and dynamic functionality.
Python	Used as the main programming language for the backend.
Flask	Used to develop the backend and REST API.
MySQL	Used to store and manage application data.
PyMySQL	Used to connect the Flask application with the MySQL database.

Technology	Purpose
Leaflet.js	Used to display bus locations on an interactive map.
OpenStreetMap	Used as the map data source.
Postman	Used to test and verify REST API operations.
Git/GitHub	Used for source-code management and project repository.

The

following technologies are used to develop the Real-Time Public Transport Tracking for Small Cities application.

5.1 Technology Roles

Frontend

HTML, CSS, and JavaScript are used to create the user interface and allow users to interact with the application.

Backend

Python with Flask handles the application logic, database communication, and REST API requests.

Database

MySQL stores user, bus, and route information.

API Testing

Postman is used to test Create, Read, Update, Delete, validation, and error-handling operations.

6. SYSTEM ARCHITECTURE

The Real-Time Public Transport Tracking for Small Cities follows a simple three-layer architecture consisting of the frontend, backend, and database.

6.1 Architecture Flow

User



HTML / CSS / JavaScript Frontend



Flask Backend & REST API



MySQL Database

- The frontend provides the user interface through which users can interact with the application. The Flask backend processes user requests and provides REST API endpoints for bus-related operations.
- The Flask backend communicates with the MySQL database to store, retrieve, update, and delete bus and route information.

6.2 Bus Location Flow

Bus Location Data (Latitude & Longitude)



Flask Backend



Map Interface



User Views Bus Location

- Latitude and longitude values are stored for buses and can be used to represent their location on the map.

6.3 Components

Frontend

HTML, CSS, and JavaScript provide the interface for users to access and interact with the system.

Backend

Flask handles application logic, REST API requests, validation, and communication with the database.

Database

MySQL stores user, bus, and route information.

API Testing

Postman is used to send requests to the REST API and verify the responses.

Map

Leaflet.js and OpenStreetMap are used to display bus location information on an interactive map.

7. DATABASE DESIGN

The application uses MySQL as the database management system. The database is named `bus_tracking` and is used to store and manage the information required by the public transport tracking system.

The main tables used in the application are:

- `users`
- `buses`
- `routes`

7.1 Users Table

The `users` table stores the details of registered users.

Field	Description
<code>id</code>	Unique ID of the user
<code>name</code>	Name of the user

Field	Description
email	Email address of the user
password	Password used for login

7.2 Buses Table

The buses table stores bus-related information.

Field	Description
id	Unique ID of the bus record
bus_number	Bus registration/identification number
driver_name	Name of the driver
route_id	ID of the associated route
latitude	Latitude coordinate of the bus
longitude	Longitude coordinate of the bus

7.3 Routes Table

The routes table stores route information.

Field	Description
route_id	Unique ID of the route

Field	Description
source	Starting location
destination	Ending location

7.4 Database Relationship

The route_id in the buses table is used to associate a bus with a route in the routes table.

Therefore:

Routes → Buses

One route can be associated with bus records through the route_id.

7.5 Database Verification

The database was verified using MySQL Command Line Client. The `SELECT * FROM buses;` query was used to confirm that bus records created and modified through the application/API were stored in the database.

8. FRONTEND DEVELOPMENT

The frontend of the Real-Time Public Transport Tracking for Small Cities application is developed using HTML, CSS, and JavaScript. It provides a simple interface through which users can interact with the public transport system.

8.1 User Registration

The registration page allows a new user to enter the required details and create an account. The registration information is stored in the MySQL database.

8.2 User Login

The login page allows registered users to enter their credentials and access the application.

8.3 Dashboard

After successful login, the user can access the main dashboard of the bus tracking system. The dashboard provides access to the available bus-related features.

8.4 Bus Management

The application provides functionality to manage bus information. Bus details include:

- **Bus Number**
- **Driver Name**
- **Route ID**
- **Latitude**
- **Longitude**

The system supports adding, viewing, updating, and deleting bus records.

8.5 Route Management

Route information can be managed using source, destination, and route ID details. Routes are associated with buses using the route ID.

8.6 Bus Location

The application stores latitude and longitude values for buses. These coordinates can be used to display the bus location on a map.

8.7 User Interaction

The frontend provides forms, buttons, and pages that allow users to perform operations and view the results returned by the backend.

The frontend communicates with the Flask backend, which processes the requests and interacts with the MySQL database.

9. BACKEND DEVELOPMENT

The backend of the Real-Time Public Transport Tracking for Small Cities application is developed using Python and Flask. It is responsible for processing user requests,

handling application logic, communicating with the MySQL database, and providing REST API services.

9.1 Flask Backend

Flask acts as the web framework for the application. It receives requests from the frontend or Postman, processes them, and sends an appropriate response.

9.2 Database Connectivity

The Flask application is connected to the MySQL bus_tracking database. The backend performs database operations to store, retrieve, update, and delete application records.

9.3 Bus Management

The backend provides operations for managing bus records. Each bus record contains information such as:

- **Bus Number**
- **Driver Name**
- **Route ID**
- **Latitude**
- **Longitude**

9.4 REST API

REST API endpoints are implemented to perform CRUD operations on bus records.

The main operations are:

- **POST – Create a new bus**
- **GET – Retrieve bus information**
- **PUT – Update existing bus information**
- **DELETE – Delete a bus record**

9.5 Validation and Error Handling

The backend validates incoming requests and provides suitable HTTP status codes for different situations.

For example:

- 201 Created – Successful creation
- 400 Bad Request – Invalid or missing input
- 404 Not Found – Requested bus does not exist

This allows the frontend and API clients such as Postman to understand the result of each request.

10. REST API IMPLEMENTATION

The application provides REST API endpoints for managing bus information. The APIs allow clients such as the frontend and Postman to communicate with the Flask backend using HTTP methods.

10.1 API Endpoints

Operation	HTTP Method	Endpoint	Purpose
Create Bus	POST	/api/buses	Creates a new bus record
Read All Buses	GET	/api/buses	Retrieves all bus records
Read One Bus	GET	/api/buses/<bus_number>	Retrieves a specific bus

Operation	HTTP Method	Endpoint	Purpose
Update Bus	PUT	/api/buses/<bus_number>	Updates an existing bus
Delete Bus	DELETE	/api/buses/<bus_number>	Deletes a bus record

10.2 Create Bus – POST

- The POST request is used to create a new bus record. The request contains bus details such as bus number, driver name, route ID, latitude, and longitude.
- After successful creation, the API returns a successful response indicating that the bus record has been created.

10.3 Read Buses – GET

- The GET request is used to retrieve bus information from the database.

The system supports retrieving:

- All available buses
- A specific bus using its bus number

10.4 Update Bus – PUT

- The PUT request is used to modify the details of an existing bus. Updated information is stored in the MySQL database.

10.5 Delete Bus – DELETE

- The DELETE request removes an existing bus record from the database.

10.6 API Response and Error Handling

The APIs return appropriate HTTP status codes based on the result of the request.

- **201 Created – Bus successfully created**
- **200 OK – Request successfully processed**
- **400 Bad Request – Invalid or missing input**
- **404 Not Found – Bus record does not exist**

The REST APIs were tested using Postman to verify their functionality.

11. CRUD IMPLEMENTATION

The Real-Time Public Transport Tracking for Small Cities application implements the four basic database operations: Create, Read, Update, and Delete (CRUD). These operations are implemented through Flask REST APIs and connected to the MySQL database.

11.1 Create Operation

The Create operation is implemented using the HTTP POST method.

A new bus can be added by providing details such as:

- **Bus Number**
- **Driver Name**
- **Route ID**
- **Latitude**
- **Longitude**

The Flask backend receives the request and stores the new bus record in the MySQL database.

11.2 Read Operation

The Read operation is implemented using the HTTP GET method.

The application provides two types of read operations:

- **Retrieve all bus records.**

- Retrieve a specific bus using its bus number.

The requested information is retrieved from the MySQL database and returned as a response.

11.3 Update Operation

The Update operation is implemented using the HTTP PUT method.

It allows the details of an existing bus to be modified. The updated information is stored in the MySQL database.

11.4 Delete Operation

- The Delete operation is implemented using the HTTP DELETE method.
- It removes the selected bus record from the database. The operation was also verified by checking the database after deletion.

11.5 CRUD Summary

CRUD Operation	HTTP Method	Function
Create	POST	Add a new bus
Read	GET	View bus information
Update	PUT	Modify bus information
Delete	DELETE	Remove a bus

The CRUD operations were tested using Postman, and the database was verified using MySQL.

12. VALIDATION AND ERROR HANDLING

Validation and error handling are implemented in the application to ensure that invalid requests are handled properly and meaningful responses are returned to the user.

12.1 Input Validation

The REST API checks whether the required information is provided in the request. If required data is missing or invalid, the API returns a 400 Bad Request response.

This prevents incomplete or invalid information from being processed.

12.2 Bus Not Found Handling

When a user requests a bus that does not exist in the database, the API returns a 404 Not Found response.

For example, requesting a non-existing bus number results in an appropriate error message indicating that the bus was not found.

12.3 HTTP Status Codes

Status Code	Meaning	Application Usage
200	OK	Successful request
201	Created	New bus successfully created
400	Bad Request	Invalid or missing input
404	Not Found	Bus record does not exist

12.4 Error Handling

The backend handles invalid requests and unavailable records without causing the application to stop unexpectedly. Appropriate HTTP status codes and messages are returned to the API client.

The validation and error-handling functionality was tested using Postman.

13. POSTMAN TESTING

- Postman was used to test and verify the REST APIs of the Real-Time Public Transport Tracking System. The API testing was performed to ensure that the Create, Read, Update and Delete (CRUD) operations work correctly.
- Each API request was tested with appropriate input data, and the response status codes and returned data were verified.

13.1 POST – Create Bus

The POST request was used to add a new bus record to the database.

- **Method: POST**
Endpoint: /api/buses

The request was tested with bus number, driver name, route ID, latitude and longitude. A successful request returned the appropriate success response.

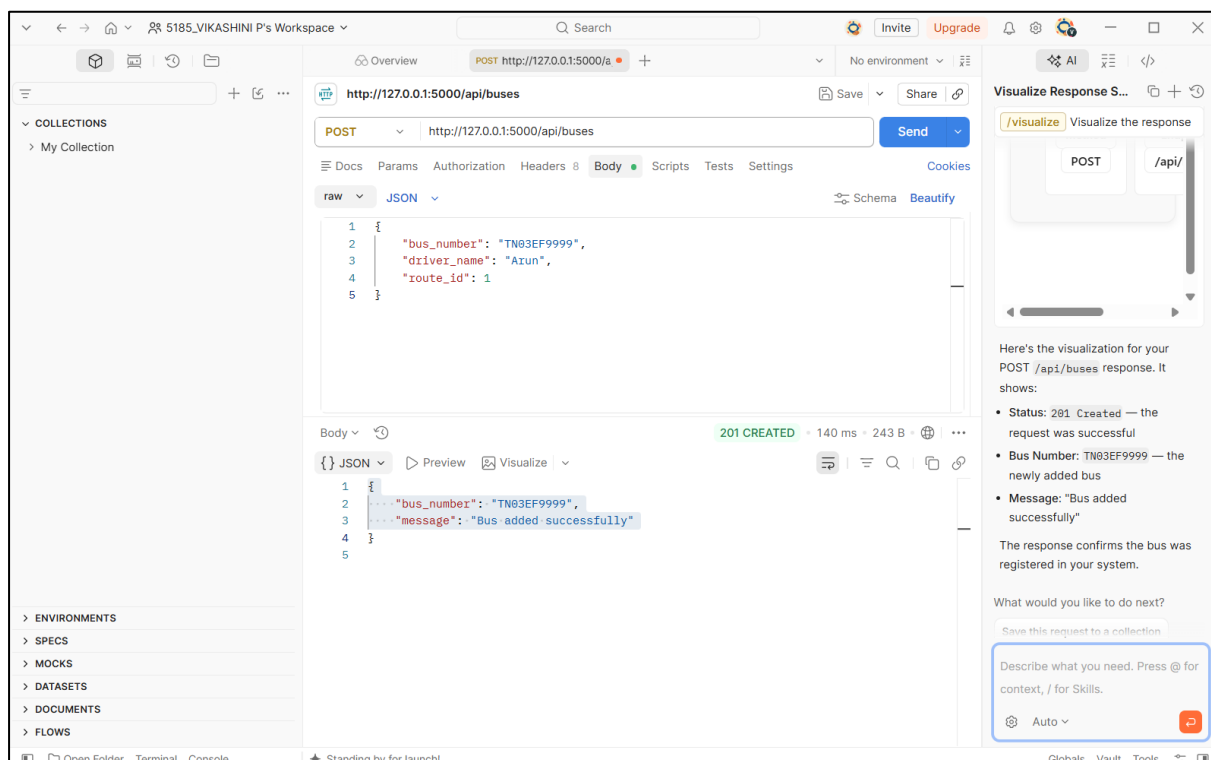


Figure 13.1: POST API – Create Bus

13.2 GET – Read All Buses

The GET request was used to retrieve all available bus records from the database.

- **Method: GET**
Endpoint: /api/buses

The response displayed the bus records stored in the database.

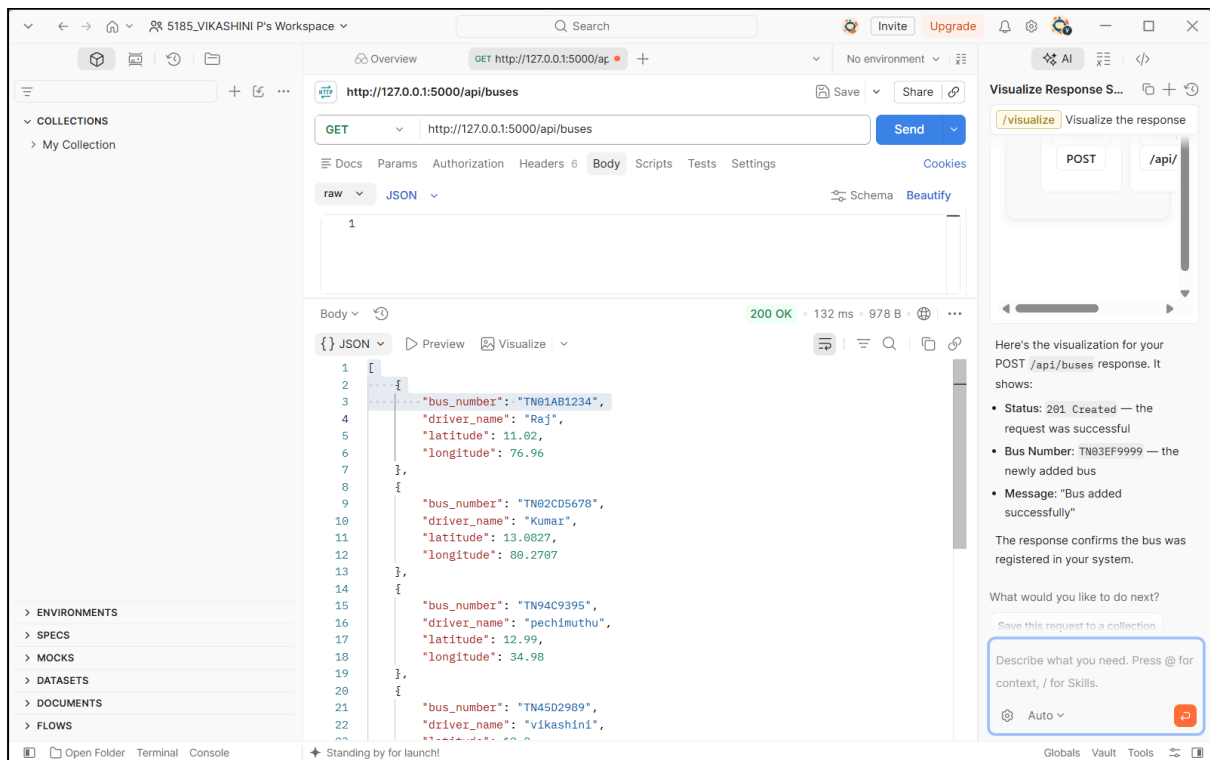


Figure 13.2: GET API – Read All Buses

13.3 GET – Read One Bus

The GET request was also tested for retrieving a specific bus using its bus number.

- **Method: GET**
Endpoint: /api/buses/<bus_number>

The response displayed the details of the requested bus.

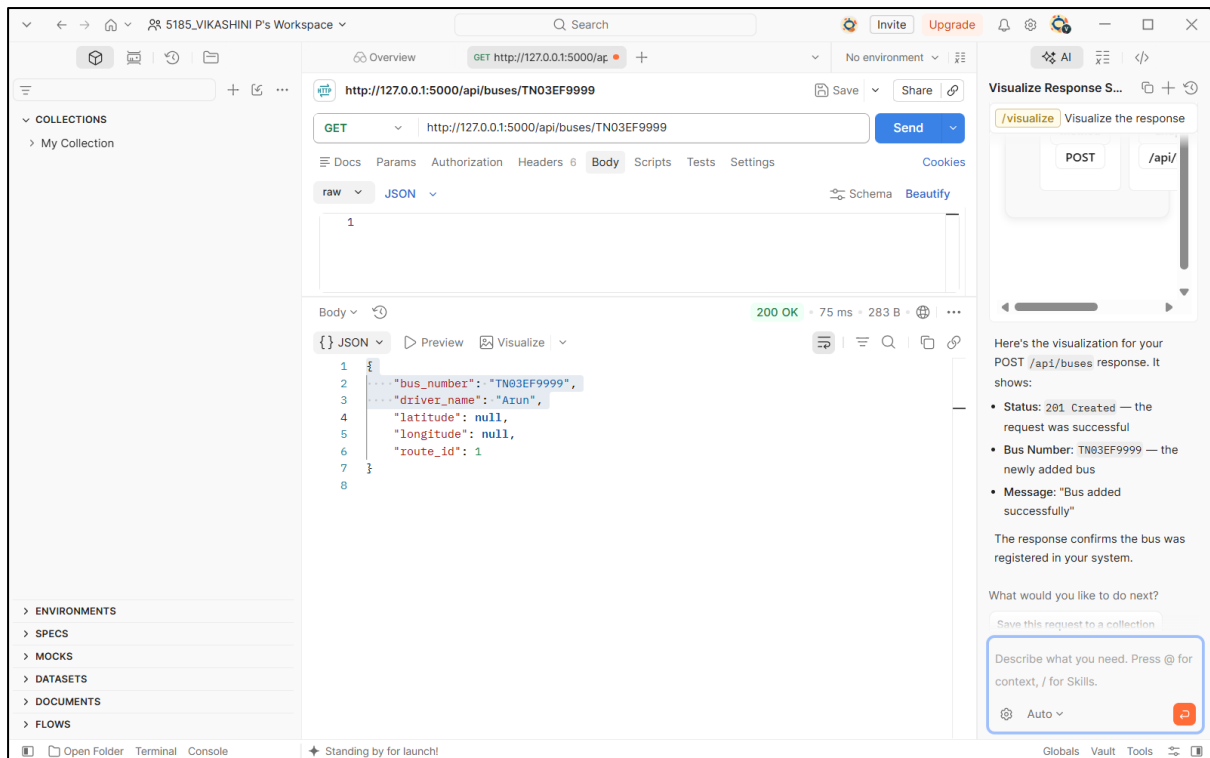


Figure 13.3: GET API – Read One Bus

13.4 PUT – Update Bus

The PUT request was used to update the details of an existing bus record.

- **Method: PUT**
Endpoint: /api/buses/<bus_number>

The updated information was sent through Postman and the response was verified.

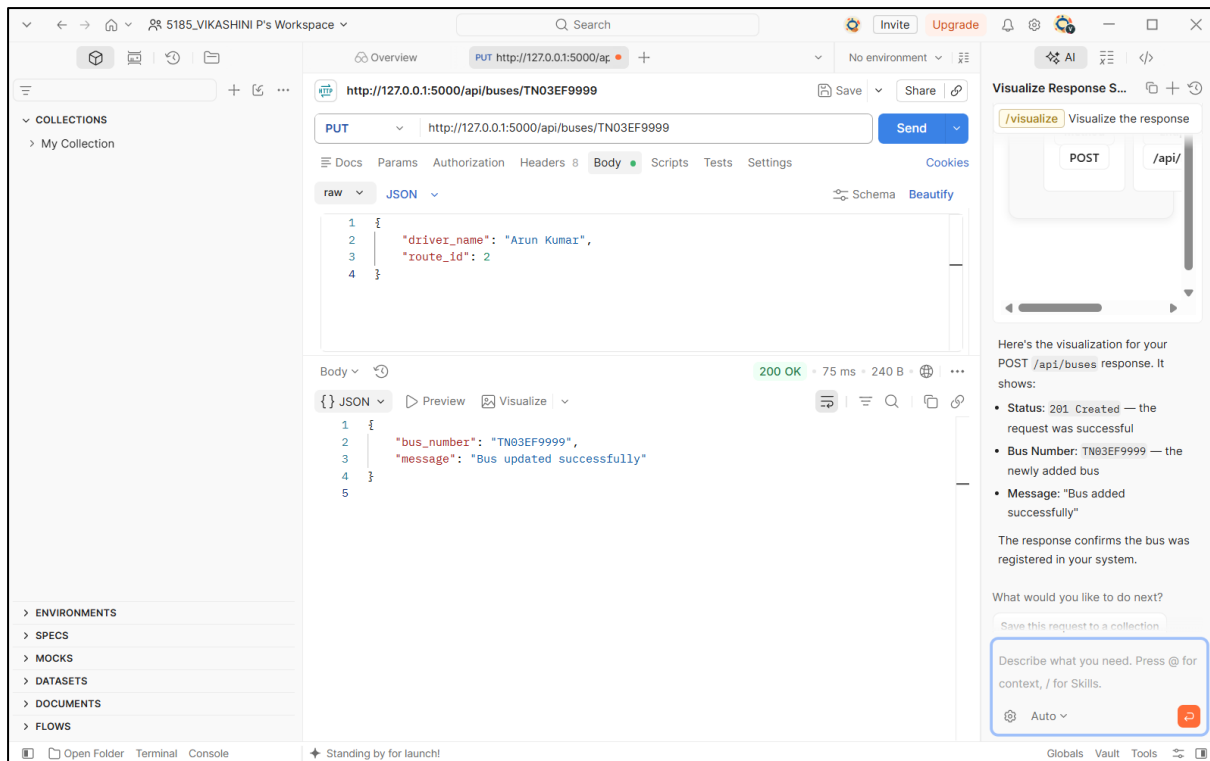


Figure 13.4: PUT API – Update Bus

13.5 DELETE – Delete Bus

The DELETE request was used to remove an existing bus record from the database.

- **Method: DELETE**

Endpoint: /api/buses/<bus_number>

The response was checked to confirm that the bus record was successfully deleted.

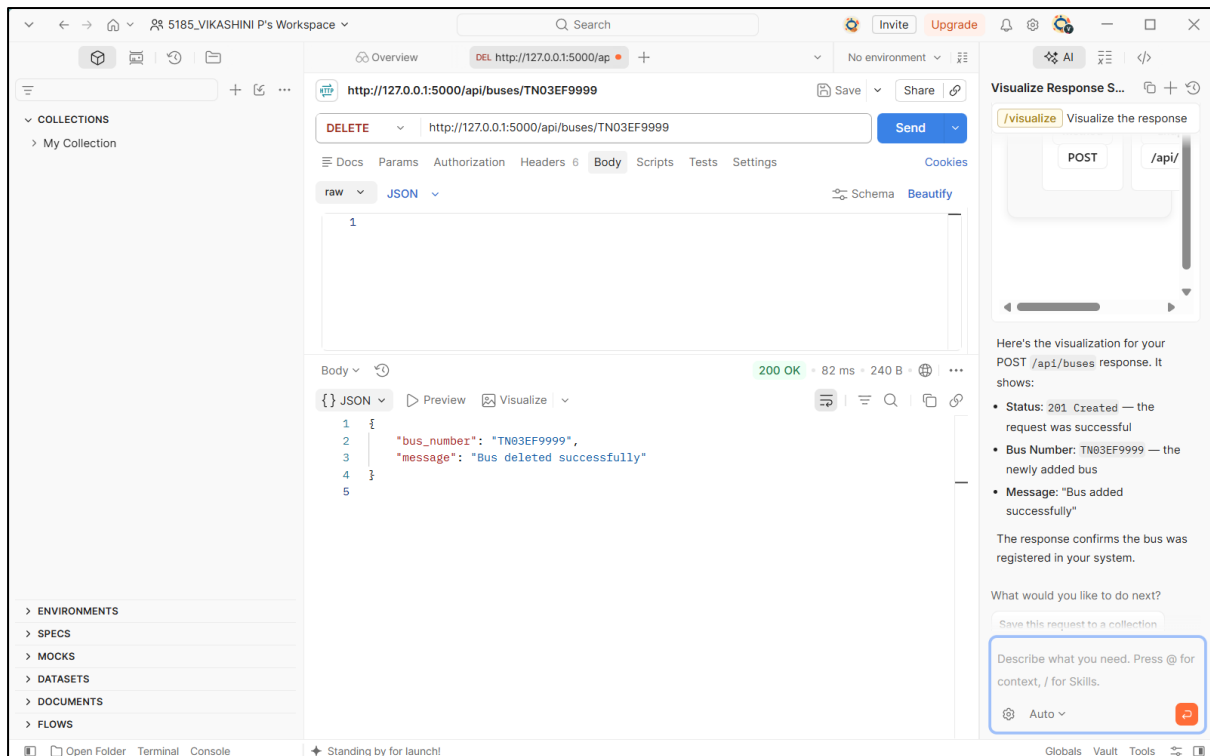


Figure 13.5: DELETE API – Delete Bus

13.6 400 – Validation Error

An invalid request was tested to verify the input validation and error-handling mechanism.

When invalid or missing input was provided, the application returned a 400 Bad Request response.

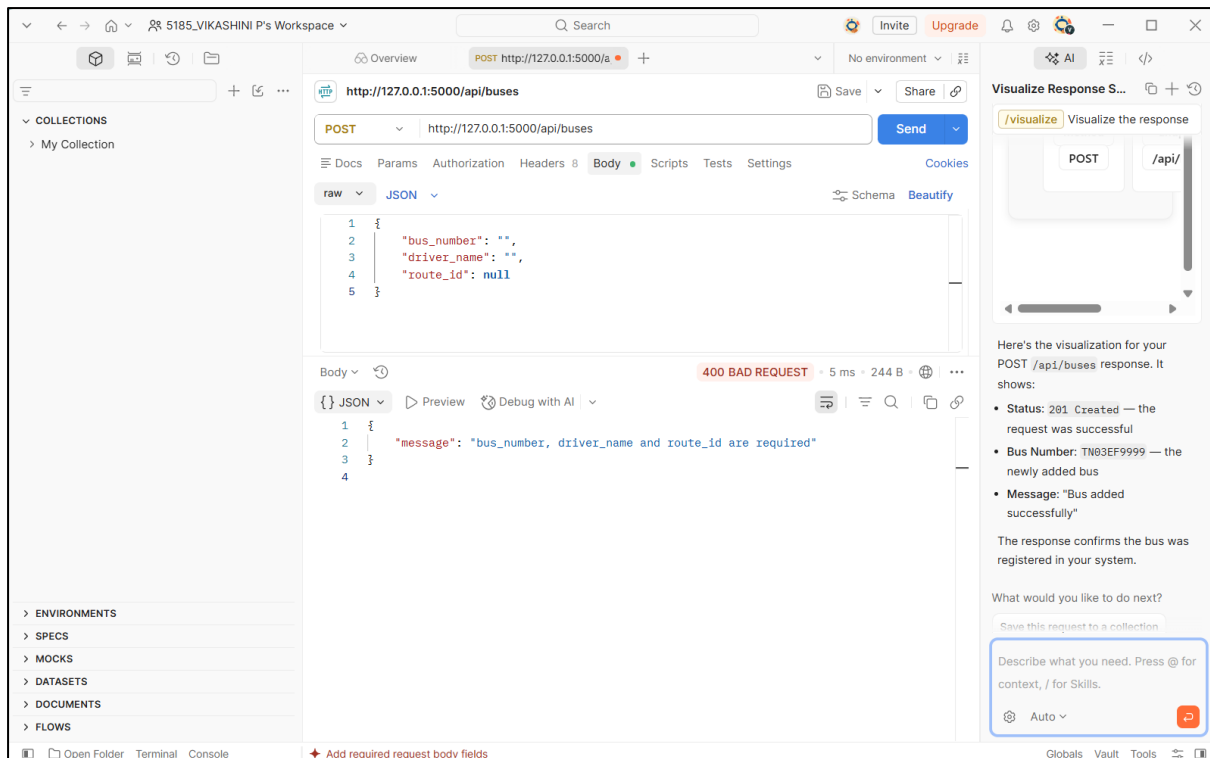


Figure 13.6: 400 Bad Request – Validation Error

13.7 404 – Not Found

A request was made for a bus that does not exist in the database.

The application returned a 404 Not Found response, confirming that the system handles unavailable records correctly.

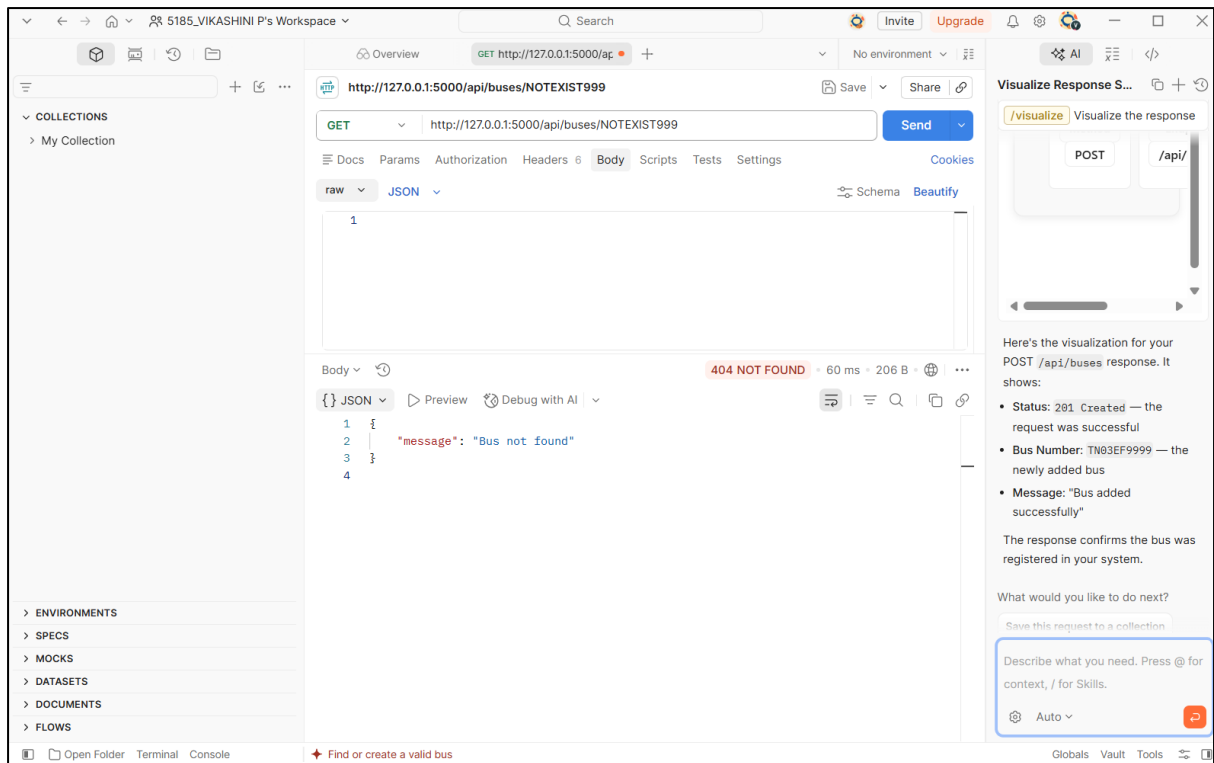


Figure 13.7: 404 Not Found – Bus Not Found

13.8 MySQL Database Verification

- After performing the API operations, the database was verified using the MySQL Command Line Client.

The following query was used to view the bus records:

- **SELECT * FROM buses;**

The output confirmed that the API operations were properly reflected in the MySQL database.

```
mysql> USE bus_tracking;
Database changed
mysql> SELECT * FROM buses;
```

id	bus_number	driver_name	route_id	latitude	longitude
1	TN01AB1234	Raj	1	11.02	76.96
2	TN02CD5678	Kumar	2	13.0827	80.2707
3	TN01AB1234	kanan	1989	NULL	NULL
4	TN94C9395	pechimuthu	1981	12.99	34.98
5	TN45D2989	vikashini	8765	12	98
6	TNP2007	yashwanth	2007	156	908
7	TN01AB1234	Raj	1	NULL	NULL
8	TN02CD5678	Kumar	2	NULL	NULL
9	TNB2006	arun	1234	345	9.8
11	TNK1989	murugan	1234	4567	2341

```
10 rows in set (0.00 sec)

mysql> |
```

Figure 13.8: MySQL Database Verification

13.9 Testing Summary

Test	Method	Purpose	Result
Create Bus	POST	Add a new bus	Passed
Read All Buses	GET	Retrieve all buses	Passed
Read One Bus	GET	Retrieve a specific bus	Passed
Update Bus	PUT	Modify bus details	Passed
Delete Bus	DELETE	Remove a bus	Passed
Validation Error	POST	Test invalid input	Passed
Not Found	GET	Test unavailable bus	Passed
Database Verification	SQL	Verify stored records	Passed

The Postman testing confirmed that the REST API and CRUD operations of the system were functioning correctly.

14. USER INTERFACE AND SCREENSHOTS

- The user interface of the Real-Time Public Transport Tracking System was developed using HTML, CSS and JavaScript. The interface provides simple navigation for users to register, log in, view the dashboard, manage buses and routes, and view bus location information.
- The major screens of the application are described below.

14.1 Registration Page

The Registration Page allows a new user to create an account by providing the required details.

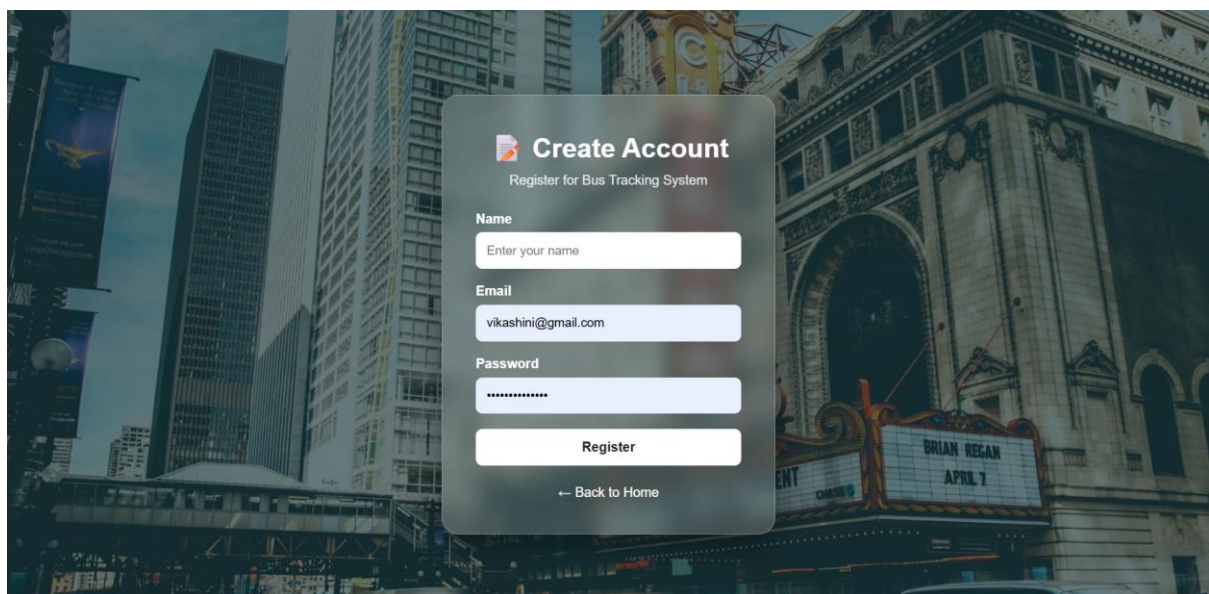


Figure 14.1: User Registration Page

14.2 Login Page

The Login Page allows registered users to enter their login credentials and access the system.

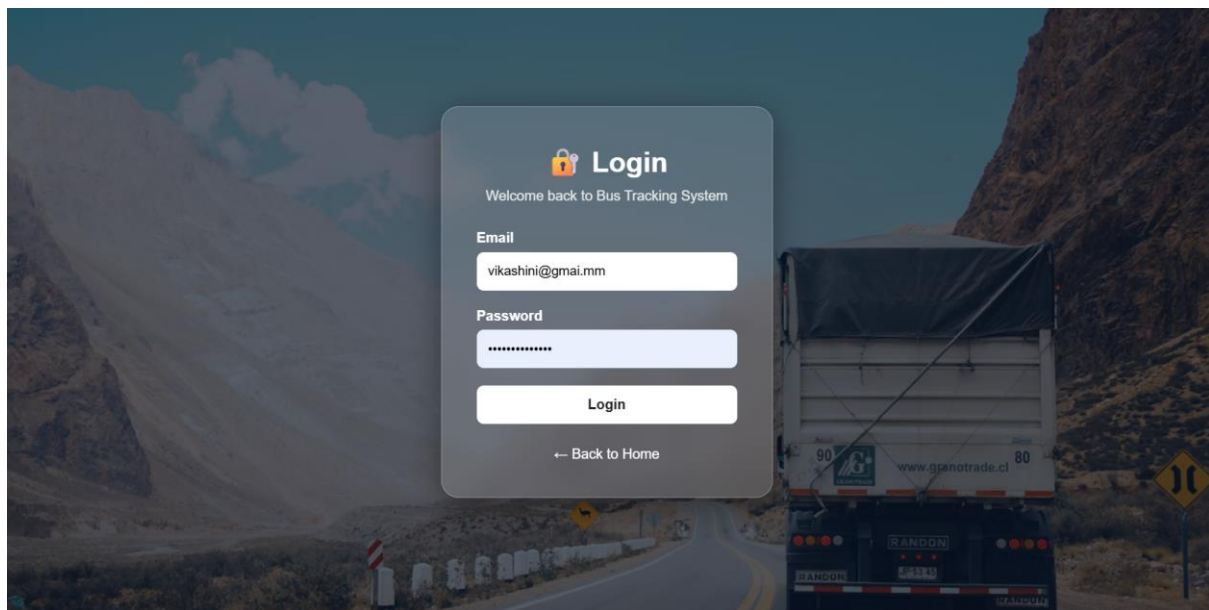


Figure 14.2: User Login Page

14.3 Dashboard

The Dashboard provides the main interface of the system after successful login. It provides access to the major functions of the application.

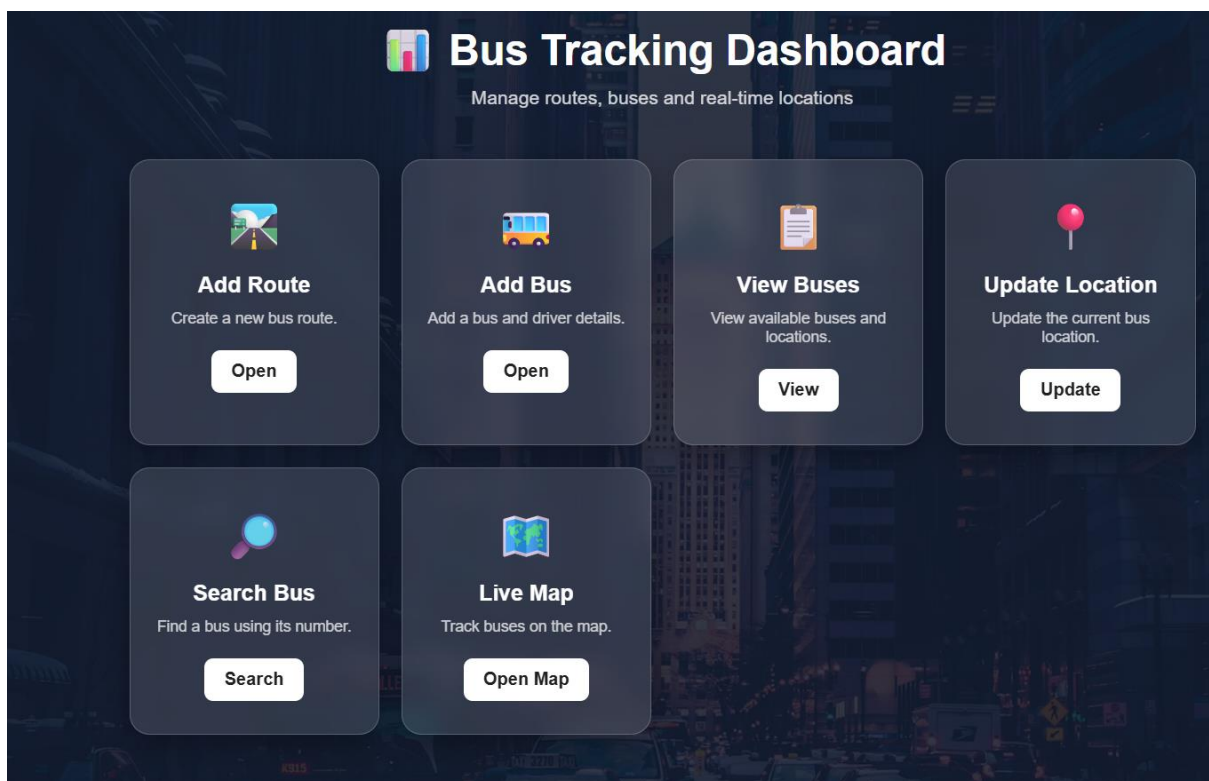
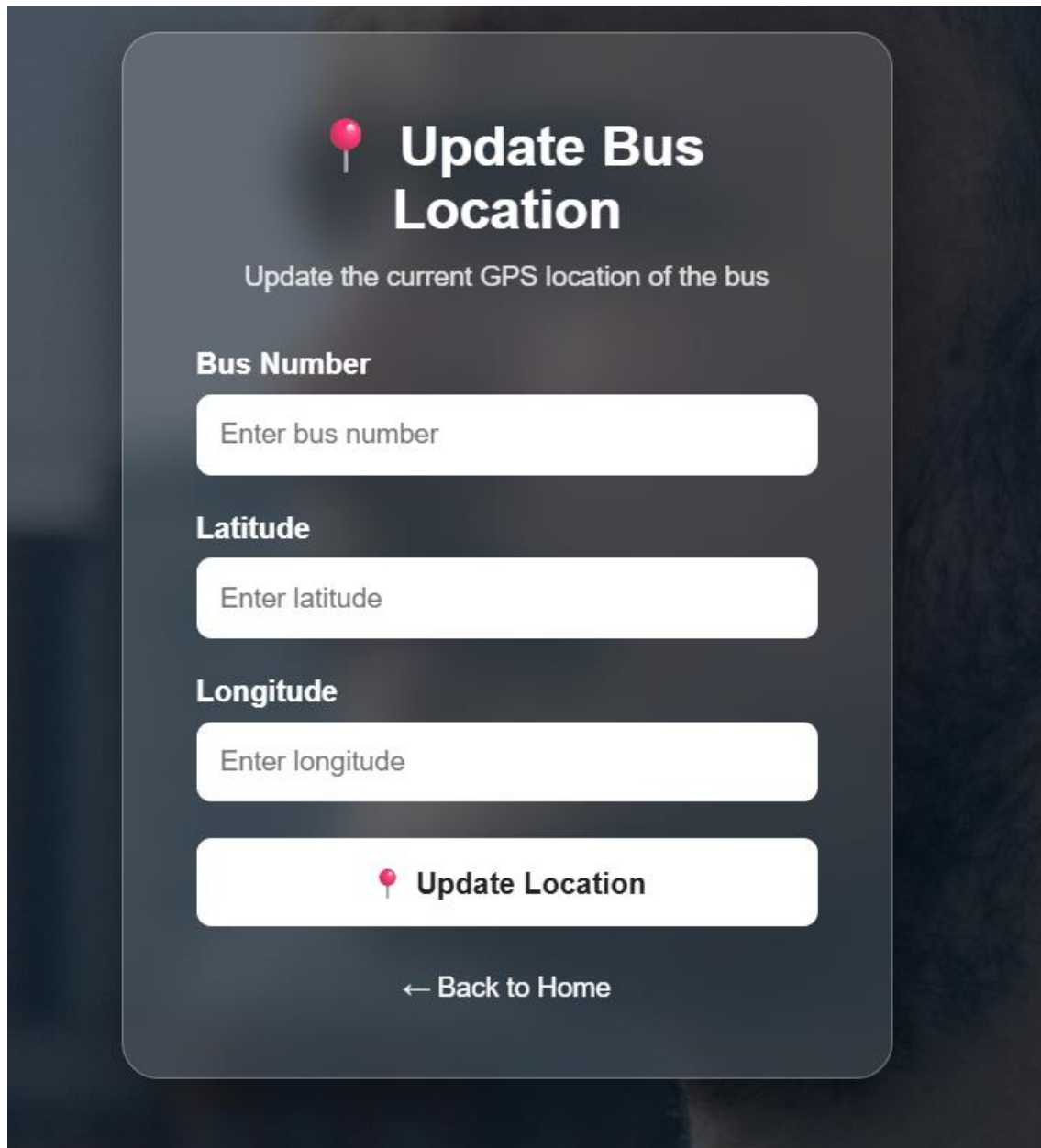


Figure 14.3: System Dashboard

14.4 Bus Management

The Bus Management interface allows the administrator to add and manage bus details such as bus number, driver name, route ID, latitude and longitude.



The image shows a mobile application interface for updating a bus location. It features a dark blue background with a central white card. At the top of the card is a red location pin icon followed by the title 'Update Bus Location' in bold. Below the title is a subtitle 'Update the current GPS location of the bus'. The form contains three input fields: 'Bus Number' with a placeholder 'Enter bus number', 'Latitude' with a placeholder 'Enter latitude', and 'Longitude' with a placeholder 'Enter longitude'. At the bottom of the card is a large white button with a red location pin icon and the text 'Update Location'. Below the button is a link '← Back to Home'.

Update Bus Location

Update the current GPS location of the bus

Bus Number

Enter bus number

Latitude

Enter latitude

Longitude

Enter longitude

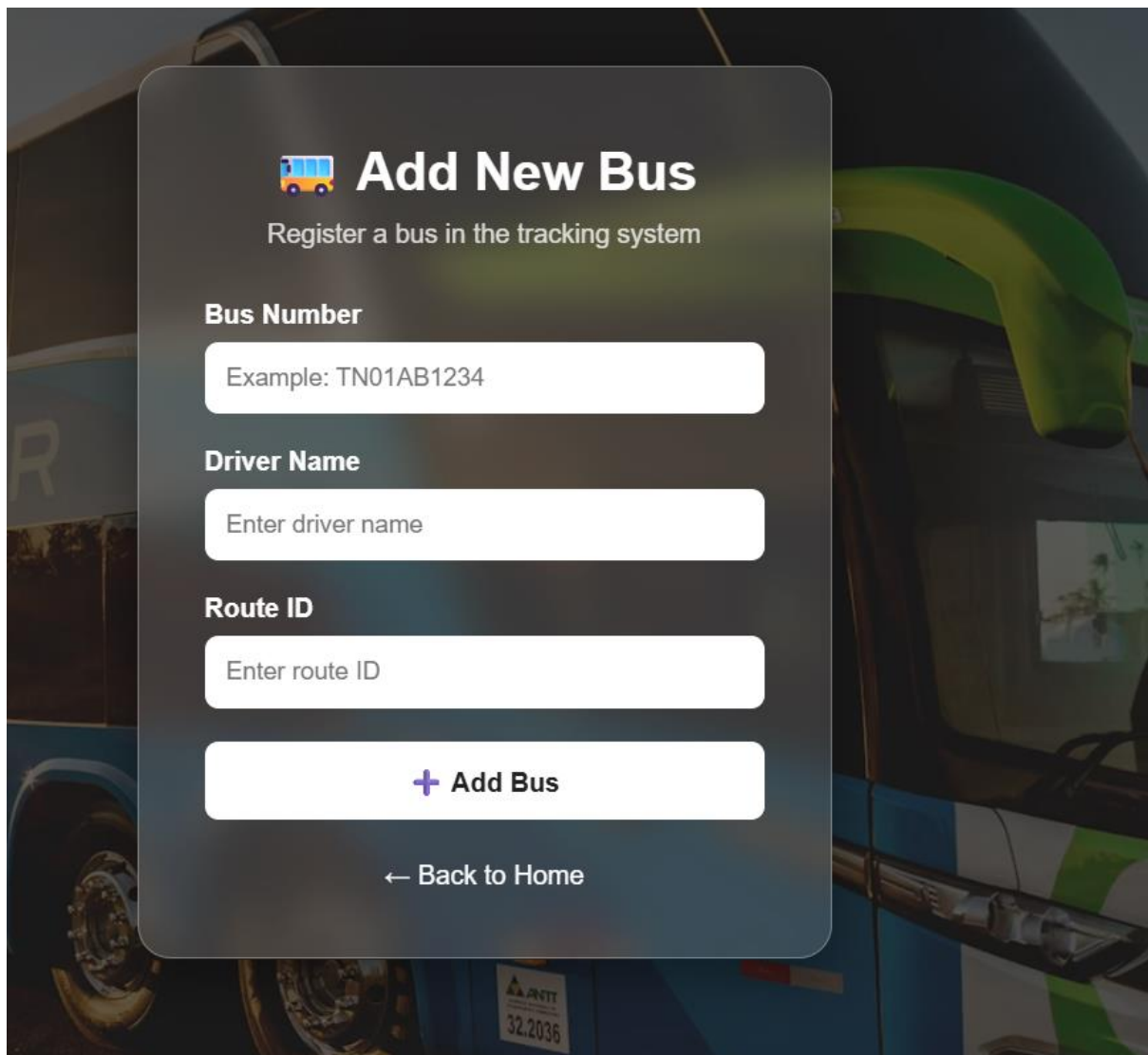
 **Update Location**

[← Back to Home](#)

Figure 14.4: Bus Management Interface

14.5 Route Management

The Route Management interface is used to manage route information, including source, destination and route details.



The image shows a mobile application interface for adding a new bus. The background is a dark, slightly blurred photograph of a blue and green bus. Overlaid on this is a semi-transparent dark grey modal box with rounded corners. At the top of the modal, there is a bus icon followed by the title 'Add New Bus' in large white font, and a subtitle 'Register a bus in the tracking system' in smaller white font. Below the subtitle, there are three input fields, each with a label in bold white text: 'Bus Number' (with a placeholder 'Example: TN01AB1234'), 'Driver Name' (with a placeholder 'Enter driver name'), and 'Route ID' (with a placeholder 'Enter route ID'). At the bottom of the modal, there is a large white button with a purple plus icon and the text '+ Add Bus', and below that, a smaller white link with a left-pointing arrow and the text '← Back to Home'.

Figure 14.5: Route Management Interface

14.6 Bus Location / Map

The Map interface displays the bus location using latitude and longitude information. It provides a visual representation of the current bus location.

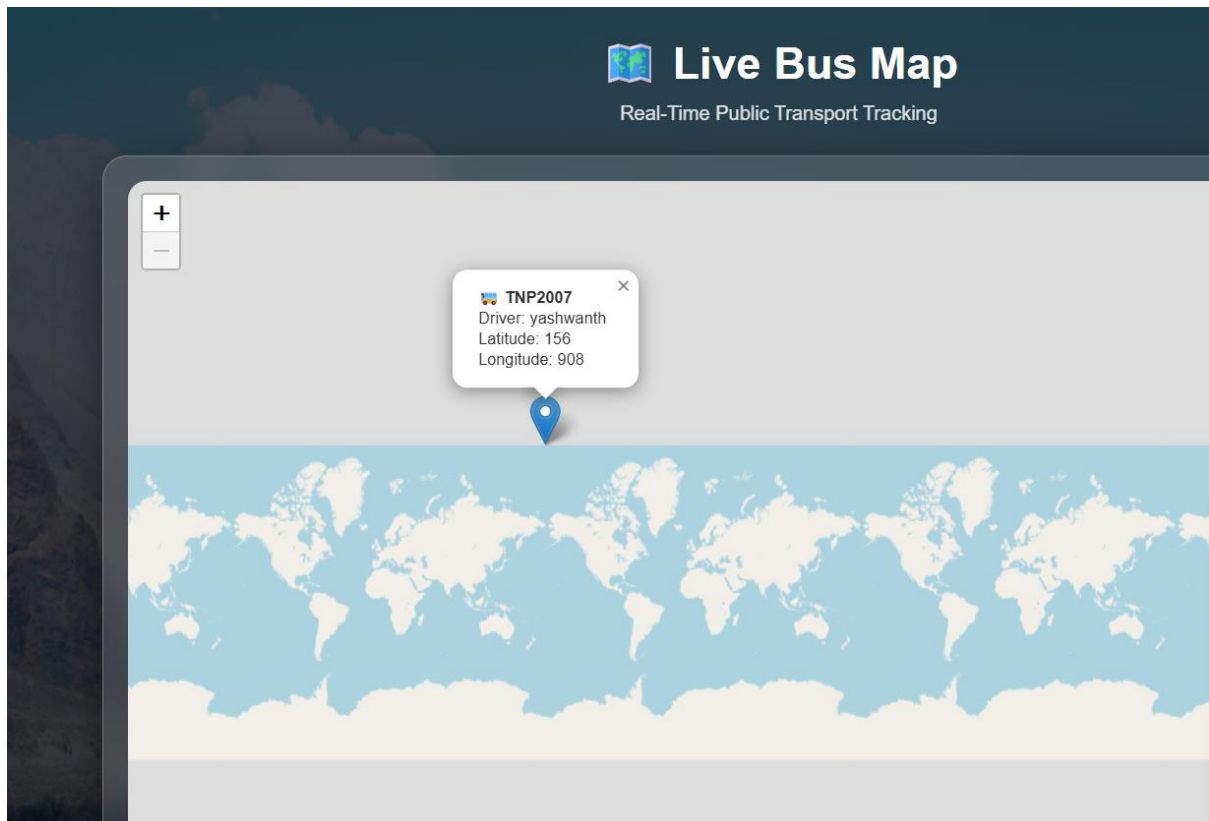


Figure 14.6: Bus Location Map

14.7 User Interaction

- The application provides a simple and user-friendly interface through which users can navigate between different functions. The interface connects the frontend with the Flask backend and database to display and manage the required information.
- Overall, the user interface is designed to make public transport tracking and bus information management simple and accessible.

15. ENTITY RELATIONSHIP (ER) DIAGRAM

The Entity Relationship (ER) Diagram represents the database structure of the Real-Time Public Transport Tracking System. It shows the main entities, their attributes and the relationship between the entities.

The database consists of three main entities:

15.1 Users

The Users entity stores the details of registered users.

Attributes:

- User ID
- Name
- Email
- Password

15.2 Buses

The Buses entity stores the details of buses used in the tracking system.

Attributes:

- Bus ID
- Bus Number
- Driver Name
- Route ID
- Latitude
- Longitude

15.3 Routes

The Routes entity stores the route information of the buses.

Attributes:

- Route ID
- Source
- Destination

15.4 Relationship

The Routes and Buses entities are related through the Route ID.

A route can be associated with bus records, and the route_id in the Buses table identifies the route assigned to a bus.

The Users entity is used for user registration and login, while the Buses and Routes entities are used for transport tracking and management.

15.5 ER Diagram Representation

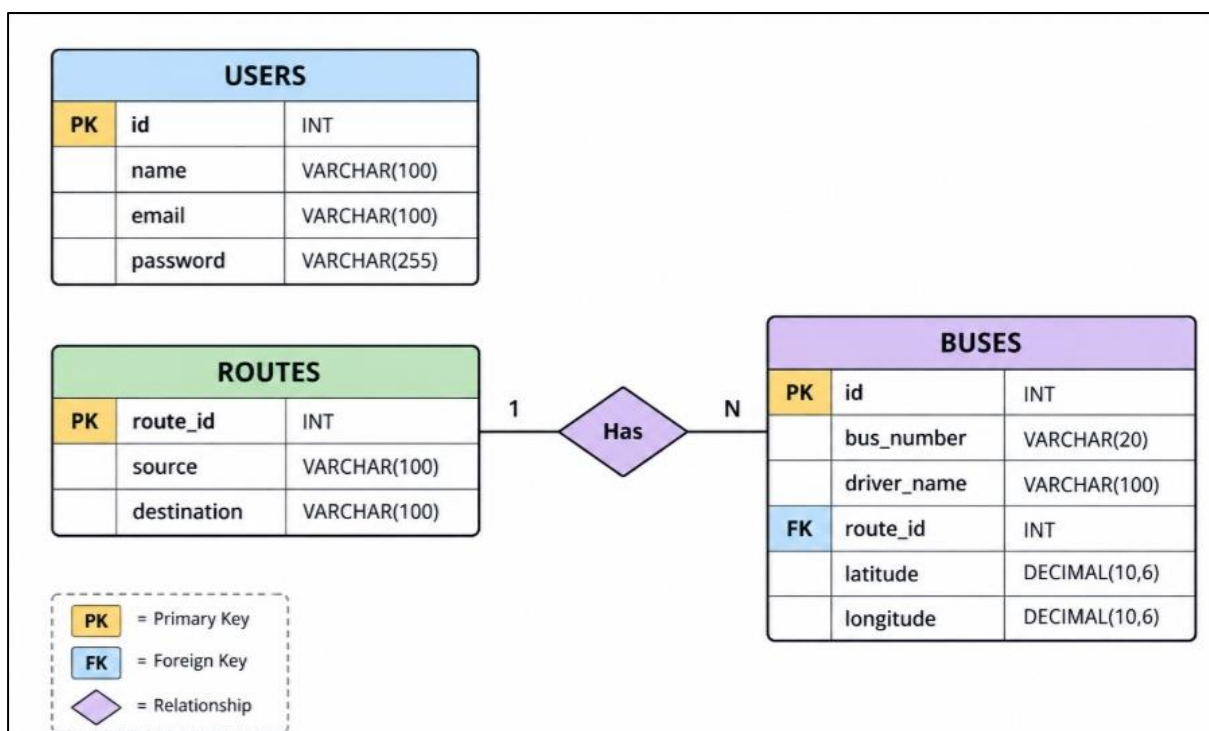


Figure 15.1: Entity Relationship Diagram of the Bus Tracking System

16. INSTALLATION AND EXECUTION PROCEDURE

The Real-Time Public Transport Tracking System can be executed on a local computer using Python, Flask and MySQL.

16.1 Software Requirements

The following software and tools are required:

- Python

- **Visual Studio Code**
- **MySQL 8.0**
- **MySQL Command Line Client**
- **Postman**
- **Web Browser**

16.2 Project Setup

1. **Open the project folder BusTrackingProject in Visual Studio Code.**
2. **Open the terminal in Visual Studio Code.**
3. **Verify that Python is installed using the command:**

```
python --version
```

4. **Install the required Python packages if they are not already installed.**

```
pip install flask pymysql
```

16.3 Database Setup

1. **Open MySQL Command Line Client.**
2. **Create or select the project database.**

```
USE bus_tracking;
```

3. **Ensure that the required tables are available:**

- **users**
- **buses**
- **routes**

4. **Verify the bus records using:**

```
SELECT * FROM buses;
```

16.4 Running the Flask Application

- 1. Open the project folder in Visual Studio Code.**
- 2. Open the terminal.**
- 3. Run the Flask application using:**

```
python app.py
```

- 4. Wait until the Flask server starts successfully.**
- 5. Open a web browser.**
- 6. Enter the local application address:**

```
http://127.0.0.1:5000
```

16.5 Application Execution

After opening the application, the user can:

- 1. Register a new account.**
- 2. Log in using the registered credentials.**
- 3. Access the dashboard.**
- 4. Manage bus details.**
- 5. Manage route details.**
- 6. View bus location information on the map.**
- 7. Perform CRUD operations through the REST API.**
- 8. Verify the stored data in the MySQL database.**

16.6 Postman Execution

- The REST APIs can be tested using Postman by sending POST, GET, PUT and DELETE requests to the Flask API endpoints.**
- The API responses are verified using HTTP status codes and returned data.**

16.7 Execution Result

The application was successfully executed in the local development environment. The Flask server, MySQL database, web interface and REST APIs were tested to verify the working of the system.

17. CHALLENGES FACED AND SOLUTIONS

During the development of the Real-Time Public Transport Tracking System, some challenges were faced while integrating the frontend, backend, database and REST APIs. The major challenges and the solutions implemented are given below.

17.1 Database Connectivity

Challenge:

Connecting the Flask application with the MySQL database and accessing the required tables was initially challenging.

Solution:

The PyMySQL library was used to establish the connection between the Flask backend and MySQL database. The database connection was tested before performing database operations.

17.2 CRUD API Implementation

Challenge:

Implementing Create, Read, Update and Delete operations through REST APIs required proper handling of requests and database queries.

Solution:

Separate API endpoints were implemented for POST, GET, PUT and DELETE operations. Each operation was tested using Postman.

17.3 Input Validation

Challenge:

Invalid or incomplete input could cause errors during bus data insertion or updating.

Solution:

Input validation and appropriate error responses were implemented. Invalid requests return a 400 Bad Request response.

17.4 Handling Missing Records**Challenge:**

A request may be made for a bus that does not exist in the database.

Solution:

The application checks whether the requested bus is available. If the record is not found, a 404 Not Found response is returned.

17.5 Frontend and Backend Integration**Challenge:**

Connecting the frontend pages with the Flask backend and database required proper routing and request handling.

Solution:

Flask routes were used to connect the frontend with backend functionality, while database queries were used to retrieve and update the required information.

17.6 Bus Location Display**Challenge:**

Displaying bus location information using latitude and longitude required integration between the stored location data and the map interface.

Solution:

Latitude and longitude values are stored in the database and used by the application to display bus location information on the map.

17.7 Testing and Debugging

Challenge:

Errors in API requests, database operations and application flow had to be identified and corrected during development.

Solution:

The application was tested using the browser, MySQL Command Line Client and Postman. Different successful and error scenarios were tested to verify the reliability of the system.

17.8 Summary

The challenges encountered during development provided practical experience in database connectivity, backend development, REST API implementation, validation, testing and frontend-backend integration. The implemented solutions helped in developing a functional CRUD-based public transport tracking application.

18. FUTURE ENHANCEMENTS

The current system provides the basic functionality required for real-time public transport tracking and bus management. The following features can be added in future versions to improve the functionality, scalability and user experience of the application.

18.1 Real-Time GPS Tracking

The system can be enhanced with continuous GPS tracking so that the location of buses can be updated automatically in real time.

18.2 Mobile Application

A dedicated Android or iOS mobile application can be developed to allow passengers to access bus information conveniently from their mobile devices.

18.3 Estimated Time of Arrival (ETA)

An ETA feature can be added to calculate and display the approximate arrival time of a bus at a particular location or stop.

18.4 Crowd Level Detection

The system can be enhanced to display the crowd level inside buses as Low, Medium or High. This can help passengers choose less crowded buses.

18.5 Push Notifications

Notifications can be introduced to inform users about bus arrivals, route changes, delays and other important transport updates.

18.6 Advanced Map Features

The map interface can be improved with route visualization, multiple bus markers, bus movement tracking and detailed stop information.

18.7 Admin Dashboard

A more advanced administrator dashboard can be developed to provide transport statistics, bus status, route information and system monitoring.

18.8 Cloud Deployment

The application can be deployed on a cloud platform so that users can access the system remotely instead of running it only in a local development environment.

18.9 Improved Security

Future versions can include stronger authentication, password encryption, role-based access control and additional security mechanisms.

18.10 Summary

These enhancements can make the system more reliable, scalable and useful for public transport users. The future development of the project can focus on real-time data, mobile accessibility, intelligent transport features and cloud-based deployment.

19. GITHUB REPOSITORY DETAILS

The source code of the Real-Time Public Transport Tracking System is maintained using Git and GitHub. GitHub is used to store and manage the project files and to maintain the project source code in a centralized repository.

19.1 Repository Contents

The GitHub repository contains the major project files and resources, including:

- Flask backend source code
- HTML templates
- CSS files
- JavaScript files
- Database-related files
- Project documentation
- README.md file
- Required project resources

19.2 Repository Management

- Git was used for version control during the development of the project. The project files were organized into a structured project folder and uploaded to the GitHub repository.
- The repository can be used to access the source code and understand the implementation of the project.

19.3 GitHub Repository Link

GitHub Repository:

[vikashini-2007 \(Vikashini\)](#)

Repository Name:

Real-Time Public Transport Tracking for Small Cities

19.4 README Documentation

- A README.md file is included in the repository. It provides information about the project, technologies used, modules, REST APIs, database, testing and future enhancements.
- The GitHub repository provides a convenient way to maintain, share and review the project source code.

20. PROJECT WORKFLOW AND DEMONSTRATION

The complete workflow of the Real-Time Public Transport Tracking System is described below. The system connects the frontend, Flask backend, REST APIs and MySQL database to provide bus management and tracking functionality.

20.1 User Registration

The user first creates an account by entering the required registration details. The registration information is stored in the MySQL database.

20.2 User Login

The registered user enters the login credentials. After successful authentication, the user is redirected to the dashboard.

20.3 Dashboard Access

The dashboard provides access to the main functions of the application, including bus management, route management and bus location information.

20.4 Bus Management

The administrator can add and manage bus information such as:

- Bus Number
- Driver Name
- Route ID
- Latitude

- Longitude

The information is stored in the buses table.

20.5 Route Management

Route details such as source and destination are maintained through the Route Management functionality. The route information is stored in the routes table.

20.6 REST API Operations

The REST API provides CRUD operations for bus records:

Create → POST

Read → GET

Update → PUT

Delete → DELETE

These operations were tested using Postman.

20.7 Validation and Error Handling

The system validates user input and handles incorrect requests using suitable HTTP status codes.

- 200 OK – Successful request
- 201 Created – Record successfully created
- 400 Bad Request – Invalid input
- 404 Not Found – Requested record does not exist

20.8 Database Verification

The data stored through the application and REST APIs is verified using the MySQL Command Line Client.

The query:

```
SELECT * FROM buses;
```

is used to verify the bus records stored in the database.

20.9 Bus Location

The latitude and longitude values associated with buses are used to display bus location information through the map interface.

20.10 Complete Demonstration Flow

The complete demonstration flow of the project is:

Registration → Login → Dashboard → Route Management → Bus Management → Bus Location → REST API Testing → Database Verification

This workflow demonstrates the complete integration of the frontend, backend, REST API and database components of the project.

21. CONCLUSION

The Real-Time Public Transport Tracking for Small Cities project was developed to provide a simple and efficient solution for managing and tracking public transport information.

The system successfully integrates a web-based frontend with a Python Flask backend, REST APIs and a MySQL database. Users can register and log in to the application, while bus and route information can be managed through the system.

The CRUD operations for bus records were implemented using REST APIs and tested using Postman. Input validation and error handling were also implemented to handle invalid requests and unavailable records. The bus location information is represented using latitude and longitude through the map interface.

The project provided practical experience in frontend development, backend programming, database management, REST API development, validation, testing and GitHub-based project management.

Overall, the developed system demonstrates how technology can be used to improve public transport information management and provide a foundation for a more advanced real-time transport tracking solution in the future.